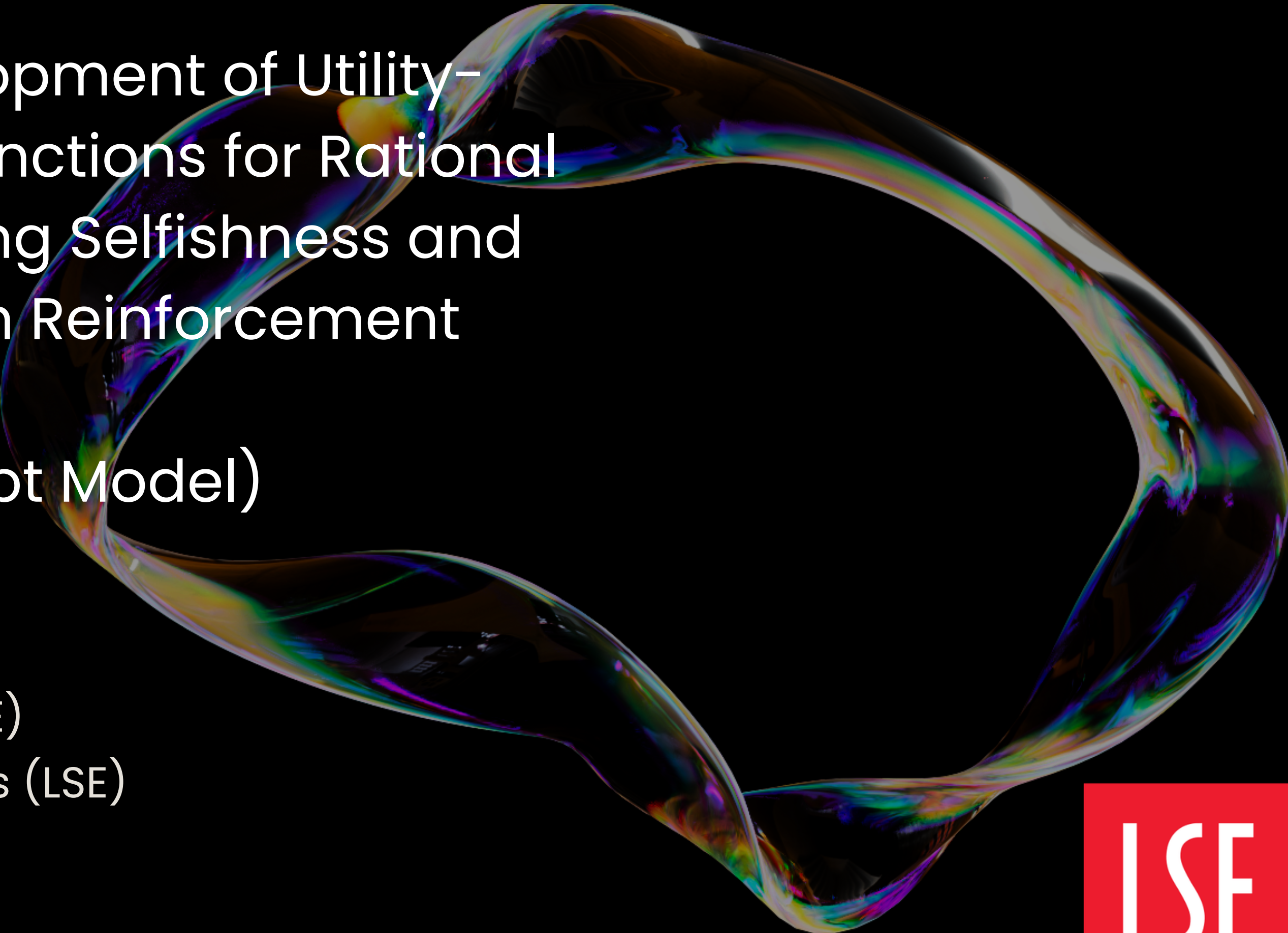




# AI-Driven Development of Utility- Maximization Functions for Rational Agents: Balancing Selfishness and Altruism through Reinforcement Learning (Proof of Concept Model)

Daria Godorozha (LSE)

Samuel Inman-Altass (LSE)



# Abstract

Rational agents maximize a given objective function, with their rationality defined by the structure of that function. The construction of such functions is inherently subjective, as different models may assign varying weights to factors like selfishness and selflessness.

For example, Model A might define rational agents as entirely selfish, while Model B introduces a weighted balance of selfishness and altruism. While subjective, the development of these functions can be informed by empirical data. The Fehr-Schmidt model (1999) incorporates both selfishness and selflessness, drawing from experimental outcomes such as Roth et al. (1991) on the ultimatum game, highlighting the importance of both self-interest and altruism in defining rational behavior.

We propose a rigorous AI driven methodology for constructing and refining objective functions of rational agents. By leveraging reinforcement learning (RL) and multi-agent simulation, we can model agent behavior under different assumptions of rationality. Specifically, we use deep RL to train simulated agents on synthetic datasets, where the agents iteratively adjust their utility-maximization strategies based on rewards defined by varying degrees of selfishness and altruism.

Our approach allows for the exploration of complex agent interactions and the emergence of nuanced behavioral patterns, advancing both the theoretical understanding and practical application of utility functions.

This methodology facilitates the rapid development of optimal utility-maximization functions from real world data, offering a scalable approach to refining economic models and agent decision-making frameworks in both theoretical and applied settings.

# Data

100% synthetic: the target behavioural moments are generated by training a population of DQN agents under the known ground truth utility function across all nine game environments, then extracting scalar behavioural statistics from the greedy evaluation tail of those agents' trajectories.

- These moments — cooperation rates, mean offers, acceptance rates, mean contributions, and mean raw rewards — constitute the target data against which candidate utility functions are evaluated.
- The process is as follows:
  - A population of agents sharing the ground truth utility is trained for 500 episodes per environment using the same DQN architecture and hyperparameters as the search candidates. The final 30% of episodes (150 episodes) form the greedy tail, during which exploration is disabled and no weight updates occur.
  - Behavioural moments are aggregated across this tail and stored as the target moment vector.
  - Candidate utility functions are then evaluated by training fresh agent populations under each candidate specification and computing the same moments from their greedy tails, with the mean squared error between candidate and target moments serving as the loss function.

This synthetic setup is used because it provides a ground truth against which recovery can be verified — the correct utility is known in advance, allowing the benchmark to assess whether the methodology can recover it. The proposal anticipates extending this to real-world behavioural data, where the same moment extraction and search procedure would be applied to moments derived from observed human or agent behaviour rather than a simulated ground truth population.

# Simulation Environments

We use nine parallel environments across five game types, all implemented as simultaneous action Markov games.

Two training environments, which are used to fit the candidate utility functions.

- Repeated Prisoner's Dilemma with two agents

At each step, both agents simultaneously choose to cooperate or defect. An agent's payoff is given by  $b$  times the opponent's action, minus  $c$  times their own action — where  $b$  is the benefit from the other's cooperation, set to 3, and  $c$  is the cost of cooperating, set to 1. Actions are binary: 1 for cooperate, 0 for defect.

This gives us four possible outcomes. If both cooperate, each gets  $3 \times 1 \text{ minus } 1 \times 1$ , which is 2. If you defect while they cooperate, you get  $3 \times 1 \text{ minus } 1 \times 0$ , which is 3. If you cooperate while they defect, you get  $3 \times 0 \text{ minus } 1 \times 1$ , which is minus 1. And if both defect, both get 0.

- Repeated Ultimatum Game with a proposer and a responder.

Each round, the proposer offers some share of a unit endowment, call it  $s$ , and the responder simultaneously accepts or rejects. If accepted, the responder receives  $s$  and the proposer keeps the remainder. If rejected, both receive nothing.

# Simulation Environments

Validation (held-out behavioural prediction error for candidate selection)

- Public Goods Game (3 agents). Each agent simultaneously chooses a contribution level from a discrete set. Total contributions are multiplied by  $r$  and split equally among all agents, creating a social dilemma between individual and collective incentives.
- Dictator Game. The proposer unilaterally allocates a fixed endowment between themselves and a passive recipient, who has no action. The proposer's chosen share,  $s$  in the closed interval zero to one, is recorded directly as the behavioural moment, making this game a clean test of distributional preferences.

Test (held out from search; used only for generalisation evaluation and significance testing)

- Shifted variants of all four training and validation games, with perturbed horizon lengths, payoff parameters, and discretisation levels, testing robustness to distributional shift.
- Sequential Trust Game (investor/trustee). The investor transfers a fraction of a unit endowment; the transferred amount is multiplied by  $r = 3.2$  and is passed to the trustee, who then returns a fraction of the resulting pool. Unlike the simultaneous games above, actions are sequential within each round, introducing a credible commitment problem absent from the training environments. This game appears exclusively in the test set, providing an out of distribution generalisation probe.

# Agent Architecture and Training

Each candidate utility function is evaluated by training a fresh agent population from scratch; no weights are shared between candidates or across restarts.

Each agent maintains an MLP Q-network with two hidden layers (64 units, ReLU), mapping environment observations to Q-values over the discrete action space. Separate active and target networks are maintained throughout training.

Algorithm: Agents are trained via Double DQN with experience replay. The TD target decouples action selection from value estimation:

$$y_t = r_t + \gamma \cdot Q^{\text{target}} \left( s', \arg \max_a Q^{\text{active}}(s', a) \right)$$

The agent's learning target at each step is the immediate reward  $r$  plus a discounted estimate of future value  $\gamma Q$ . The key feature of Double DQN is that the next action is chosen by the active network but its value is evaluated by the more stable target network, decoupling selection from evaluation to reduce overestimation bias.

We used a replay buffer of capacity 6000, minibatch size 128, uniform sampling, and a target network updated at set step intervals. Loss is Huber; gradients are clipped at 5.0. Optimisation uses Adam ( $\text{lr} = 10^{-3}$ ) with mixed-precision training on GPU.

# Reward Signal

**Reward signal.** Agents are not trained on raw game payoffs. Instead, each agent's reward at each timestep is its assigned utility function evaluated on the observed payoff vector. Under a social tradeoff utility:

$$u(r_i, \bar{r}_{-i}) = r_i + \alpha \cdot \bar{r}_{-i} \quad (1)$$

where  $r_i$  is the agent's own payoff,  $\bar{r}_{-i}$  is the mean payoff of co-players, and  $\alpha$  is the altruism weight. For example, with  $\alpha = 1.0$ , an agent observing  $r_i = 0.7$  and  $\bar{r}_{-i} = 0.5$  receives utility reward:

$$u = 0.7 + 1.0 \times 0.5 = 1.2 \quad (2)$$

Agents therefore learn policies that are rational with respect to their assigned preferences rather than raw monetary payoffs.

# Exploration and Greedy Tail Evaluation

Exploration: Linear  $\epsilon$ -greedy decay 0.05, with the decay horizon tuned to ensure convergence before the greedy evaluation phase.

Episodes are split between a training phase (epsilon-greedy with weight updates) and a greedy evaluation tail.

Moments are computed only from the tail, so they reflect the converged policy. The tail fraction was between .3 (150 greedy episodes out of 500 total, full convergence via epsilon decay = 1500)).

# Behavioural

Five scalar statistics are extracted per environment from the greedy evaluation tail. Moments that are structurally undefined in a given game, for example, acceptance rate in the Prisoner's Dilemma, are excluded from the loss rather than penalised:

| Moment            | Definition                      | Environments        |
|-------------------|---------------------------------|---------------------|
| Cooperation rate  | Fraction of cooperative actions | RPD, Trust          |
| Mean offer        | Mean share offered              | Ultimatum, Dictator |
| Acceptance rate   | Fraction of offers accepted     | Ultimatum           |
| Mean contribution | Mean contribution fraction      | PGG, Trust          |
| Mean raw reward   | Mean per-step monetary payoff   | All                 |



# Loss and Selection Criterion

Candidate utility functions are evaluated against target moments via mean squared error, summed only over moment-environment pairs where both predicted and target values are defined:

$$\mathcal{L} = \frac{1}{n} \sum_k (\hat{m}_k - m_k)^2$$

where  $\hat{m}_k$  and  $m_k$  are predicted and target values for moment  $k$ , and the sum excludes any pair where either value is undefined. Undefined moments are omitted rather than penalised, ensuring heterogeneous environments do not incur spurious losses for structurally absent statistics.

Candidates are ranked by a regularised validation objective:

$$score = \mathcal{L}_{val} + \lambda|\theta|$$

where  $\mathcal{L}_{val}$  is the validation mean squared error, the absolute value of  $\theta$  is the number of non-zero terms in the utility specification, and  $\lambda$  equals 0.02. This complexity penalty ensures a two-term candidate must achieve a validation improvement of at least 0.02 over any competing one-term candidate to be preferred, guarding against overfitting to the validation environments.

# Search Methods

- Beam search constructs utility specifications by expanding a library of symbolic expressions, base features and unary transformations of own payoff, others' mean payoff, and derived inequality terms across a discrete weight grid. At each depth, proposals are scored on training environments and the top candidates are retained for further expansion. Each candidate's final selection score is computed on held-out validation environments plus the complexity penalty.
- Evolutionary search maintains a population of 8 symbolic utility specifications over several generations. The initial population is seeded with three theoretically motivated candidates (pure selfishness, pure altruism, and the target social tradeoff form) alongside randomly initialised specifications. The top third of each generation are retained as elites; the remainder are produced by mutation, which randomly adds, removes, or perturbs weight entries.
- Exhaustive reference search evaluates compositional candidates plus three fixed baselines (selfish, social tradeoff, inequity aversion), totalling 31 candidates. This constitutes a finite restricted hypothesis class and serves as a reference benchmark; its winner is reported separately and is not eligible for primary selection.

# Search Methods

## Removed methods:

- A neural utility search and a genetic programming search were included in early runs. Both were subsequently removed: the neural method was structurally excluded from winning by its complexity penalty floor ( $0.0001 \times 497 \approx 0.050$ , exceeding best symbolic scores of  $\sim 0.020$ ), and the GP method consistently produced high complexity expressions that scored worse than simple symbolic candidates after regularisation.

# Selection Procedure and Significance Testing

The top-scoring candidates from beam and evolutionary search are ranked by regularised validation score. The top k rich-class candidates, together with the exhaustive reference winner, are advanced to a finalist pool for repeated held-out evaluation, where k is a predefined finalist count.

Repeated test evaluation: Each finalist is independently re-trained and evaluated on the 5 test environments (games) across finalist repeats different random seeds. This re-training is fresh each time, no weights are carried over. The candidate with lowest mean repeated test loss across all seeds becomes the final selected utility function.

Pairwise significance: The selected candidate is compared against each comparator via a one-sided sign test over matched seed pairs.

Bootstrap significance: Independent evaluations of the selected candidate and each comparator on the test suite, using the same random seeds for paired comparison. Bootstrap uses training + validation environments rather than test, providing an independent significance estimate.

# Results

## Significance

The winner beat all three comparators at the floor p-value of 0.048 in both pairwise and bootstrap tests – significance confirmed against the entire finalist field.

Win rate: 20/20 (100%)

Mean repeated test loss: 0.017, compared to 0.057–0.064 for all comparators.

The confidence interval (0.012, 0.022) is tight, reflecting the low variance across seeds: CV (coefficient of variation) of 0.68.

# Results Cont...

## Initial search ranking anomaly

The winner ranked third in the initial search by regularised validation score (0.168), behind two single-term candidates: one with utility weight 1.0 (score: 0.020) and one with utility weight 0.278 (score: 0.020). This is because the complexity penalty of 0.02 per term put the two-term winner at a structural disadvantage during search:

- Two-term winner:  $0.128 + 0.02 \times 2 = 0.168$
- Single-term candidates:  $0.000 + 0.02 \times 1 = 0.020$

The winner was nonetheless included in the finalist pool as the top rich-class candidate from beam search, and won comprehensively in repeated testing.

This illustrates the importance of the two-stage selection procedure: the complexity-penalised search score correctly controls overfitting during search, but the repeated test stage recovers the genuinely better predictor.

# Results Cont...

The exhaustive reference is the best utility function found by an exhaustive grid search over a restricted hypothesis class.

It serves as a known benchmark: the best a simple, interpretable, one-term linear utility can achieve within a predefined grid, without search or optimisation. It is reported separately from the primary selection because it comes from a more restricted class than beam and evolutionary search, which can discover arbitrary symbolic transformations, multi-term combinations, and continuous weights. Comparing the primary winner against the exhaustive reference quantifies how much the richer search adds over straightforward enumeration of simple candidates.

The exhaustive reference search, which enumerates single-term linear combinations, identified a scaled social feature as the best-fitting restricted-class utility, correctly recovering the social direction of the target though with an approximated weight.

The primary selection winner, a two-term specification combining own-payoff and social components, achieved substantially lower mean test loss (0.017 versus 0.058,  $p = 0.048$ ), demonstrating that the richer symbolic search adds meaningful predictive value beyond simple enumeration.

Notably, all three search methods independently converged on the social feature as the key predictive component, with the richer search additionally recovering the correct weight and the complementary contribution of the self-interest term.

# Results Cont...

The correct utility structure is recovered, all comparators are beaten at significance, and the result is stable across seeds.

# Issues/Observations

Agent convergence: The dominant early problem was that DQN agents were nowhere near convergence when moments were scored. With epsilon decay = 4000 and 120 training episodes, agents were 79% random. Behavioral moments from a nearly-random policy carry no utility-relevant signal, they reflect exploration noise rather than preference-driven choice.

- Increasing episodes to 500 helped, but ultimatum and dictator still had  $\varepsilon \approx 0.29$  at scoring time. An epsilon decay = 2000 fix resolved this, and a greedy tail reinforced it by excluding early exploratory episodes from moment computation entirely.

Target quality: Even with well-converged search candidates, if the target moments themselves were generated from under trained agents, the discrimination signal was absent. A social tradeoff agent trained to 160 episodes with high epsilon gives near random dictator allocations.

- Raising target generation to 500 episodes with greedy tail scoring worked in the sense that the target moments now genuinely reflected social preferences.

# Issues/Observations Cont.

The central scientific observation is that the methodology succeeds when the target utility function produces behavioural moments that are genuinely discriminative across the game environments used for candidate selection.

- With altruism weight = 0.35, the social tradeoff utility prescribes the same optimal actions as a selfish utility in dictator, ultimatum, and the public goods game: the validation environments produce near-identical moments regardless of which utility is being evaluated, making recovery impossible in principle. Raising altruism weight to 1.0 crosses the threshold at which the social utility prescribes qualitatively different optimal behaviour in those same environments, and the benchmark immediately recovers the correct utility structure with 100% win rate and significance against the full finalist field.

This demonstrates that the pipeline itself is sound: the RL training, moment extraction, search methods, and selection procedure all function as intended, and that the binding constraint on recovery is not computational but informational: the game environments used for validation must be chosen, and the target utility parameterised, such that different utility hypotheses generate observationally distinct behavioural signatures in those environments.

# Conclusion

The results provide proof of concept for the proposed methodology.

The abstract argues that utility function construction, while inherently subjective, can be systematically informed by empirical data, in the same way that Fehr-Schmidt drew on ultimatum game experiments to calibrate the relative weights of selfishness and altruism.

The benchmark operationalises this directly: agents are trained under candidate utility specifications and their resulting behaviour is compared against a behavioural target, with the specification that best predicts observed behaviour selected as the recovered utility function. The successful recovery is algebraically equivalent to the true social tradeoff utility, with full significance against all comparators and a 100% win rate across repeated evaluations, demonstrates that the proposed AI-driven pipeline can identify the correct balance of selfishness and altruism from behavioural data alone, without requiring the researcher to specify that balance in advance.

The benchmark progression further validates the proposal's framing of this as a scalable and rigorous methodology. The two-stage selection procedure: complexity-penalised search followed by repeated held-out evaluation with formal significance testing. Ablations will also be provided in the final paper.

The finding that all three search methods independently converged on the social component of the utility as the dominant predictive signal, and that the result held across 20 independent repeated evaluations, is robust with regards to the rapid development of optimal utility functions from real world data.

The key practical condition surfaced by the benchmark — that the environments used for evaluation must be chosen to produce observationally distinct behaviour under different utility assumptions, reads directly onto the proposal's grounding in the experimental economics tradition, where environment design is understood to be theoretically prior to inference.